
Angelo Verlain Shema

sav@gnome.org

Matrix ID: @angeloverlain:matrix.org

Add Debug Adapter Protocol Support to GJS

DESCRIPTION

GJS is a JavaScript runtime built on [Firefox's SpiderMonkey JavaScript engine](#) and the [GNOME platform libraries](#). It allows the use of the GNOME platform libraries in JavaScript programs. GJS powers GNOME Shell, Maps, Characters, Sound Recorder and many other apps.

The **Debug Adapter Protocol (DAP)** defines the abstract protocol used between a development tool (e.g. IDE or editor) and a debugger. DAP makes it possible to implement a generic debugger for a development tool that can communicate with different debuggers via Debug Adapters. DAP is already supported by the most prominent code editors and tools like VS Code, Zed, Emacs, etc... (<https://microsoft.github.io/debug-adapter-protocol/implementors/tools/>)

Today, you can already debug applications and libraries written for/with Android, [Node.js](#), Deno, C, C++, Rust, Python and more (<https://microsoft.github.io/debug-adapter-protocol/implementors/adapters/>). This proposal aims to make GJS applications and libraries like GNOME Shell, Maps, Decibels etc... debuggable by implementing a DAP adapter for GJS.

Mentor(s)

1. Philip Chimento (@pchimento)

What city and country will you reside in during the summer?

Kigali, Rwanda

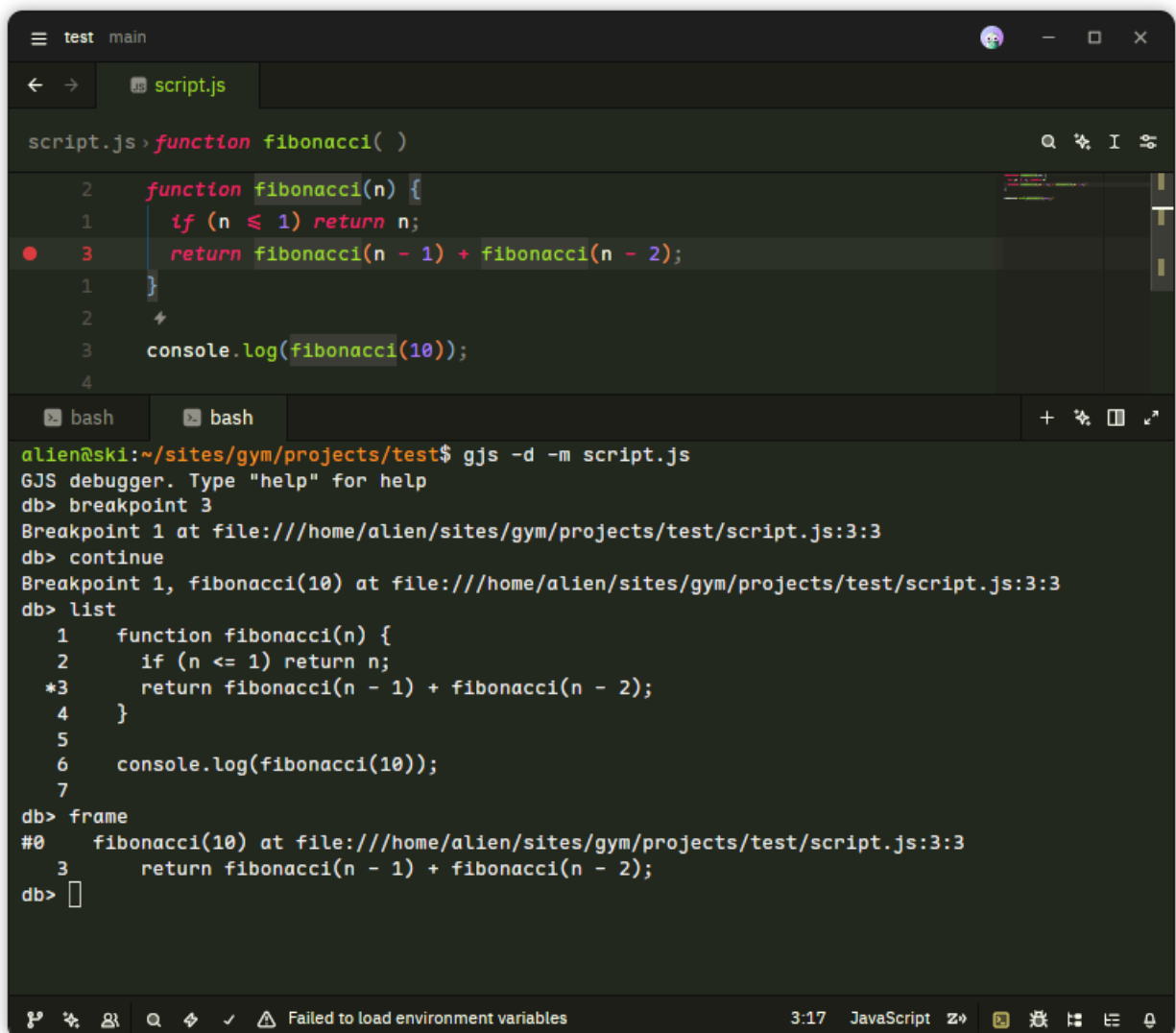
What applications/libraries of GNOME will the proposed work modify or create?

GJS: <https://gitlab.gnome.org/GNOME/gjs/>

What benefits does your proposed work have for GNOME and its community?

Debugging GJS applications will become much easier. For example, GJS already has a mostly-unknown debugger, but not many people use it, mainly because it's not graphical and interactive. GNOME Shell also has it's own "Debugger" called Looking Glass.

With DAP supported in GJS, developers will be able to click "Run" in their favourite editor and get a full debug interface with variables, traces, breakpoints, etc... displayed visually instead of running GJS with "-d" everytime they want to debug their application.



```
test main
script.js
script.js>function fibonacci( )
2 function fibonacci(n) {
1   if (n <= 1) return n;
● 3   return fibonacci(n - 1) + fibonacci(n - 2);
1 }
2
3 console.log(fibonacci(10));
4

bash bash
alien@ski:~/sites/gym/projects/test$ gjs -d -m script.js
GJS debugger. Type "help" for help
db> breakpoint 3
Breakpoint 1 at file:///home/alien/sites/gym/projects/test/script.js:3:3
db> continue
Breakpoint 1, fibonacci(10) at file:///home/alien/sites/gym/projects/test/script.js:3:3
db> list
1 function fibonacci(n) {
2     if (n <= 1) return n;
*3     return fibonacci(n - 1) + fibonacci(n - 2);
4 }
5
6 console.log(fibonacci(10));
7
db> frame
#0 fibonacci(10) at file:///home/alien/sites/gym/projects/test/script.js:3:3
3 return fibonacci(n - 1) + fibonacci(n - 2);
db>
```

Fig 1: Debugging manually using `gjs -d`

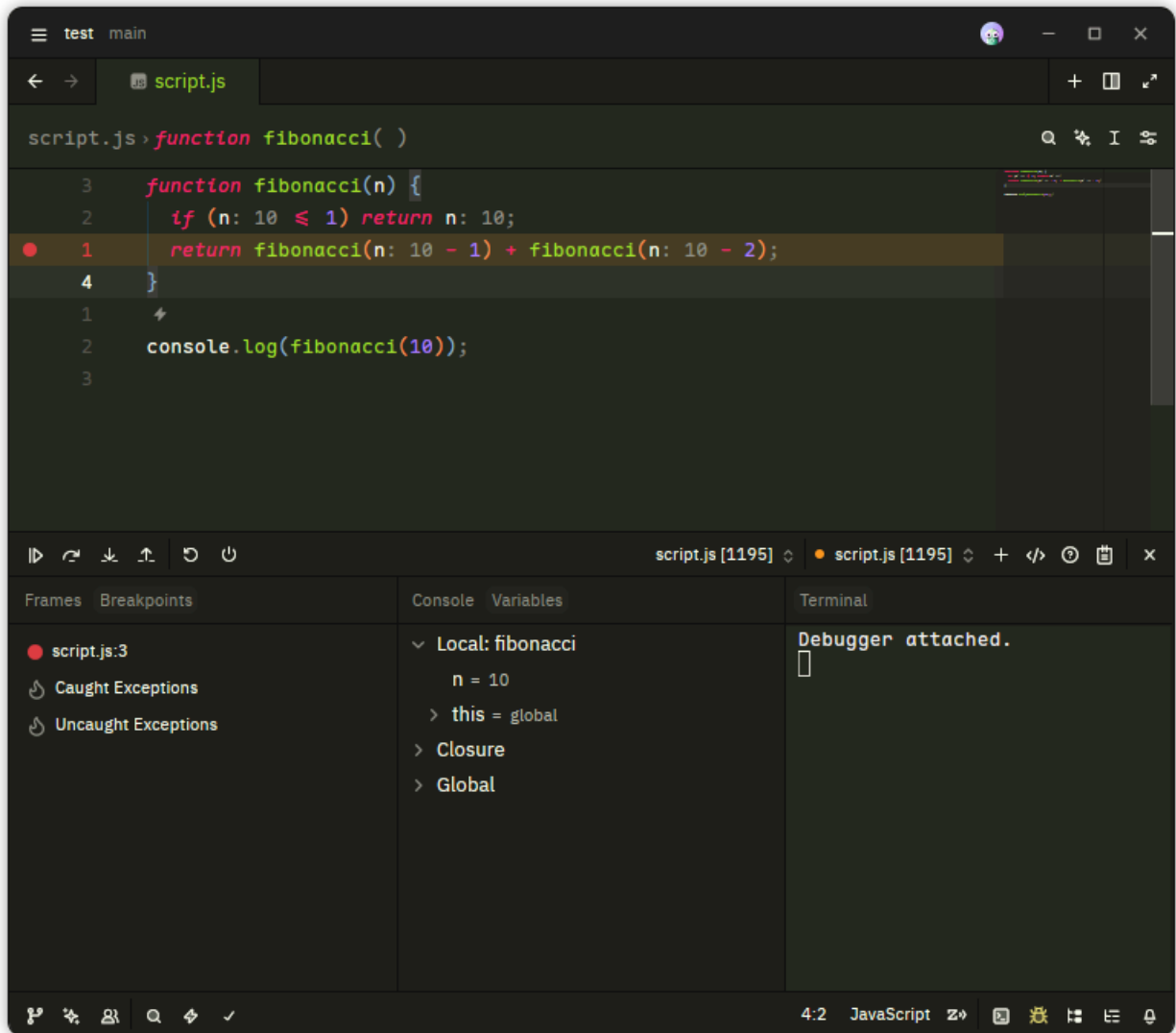


Fig 2: Debugging visually in the Zed Editor (running under [Node.js](#) since GJS doesn't support visual debugging).

Why are you the right person to work on this project?

I am the right person to work on this because I've been an open-source contributor for some time now and have contributed to the GNOME project by creating new apps and tools. I want to work on something utilised by many GNOME contributors, bringing even more developers into the project and reducing the friction in developing GNOME applications.

Furthermore, I have notable experience in the project's requirements. I am familiar with Javascript, GJS, C, GObject, and GTK. I am also a Web Developer, which means I am familiar with TypeScript and have witnessed how visual debugging makes the development flow easier and

helps us write more robust and correct code faster. I wish to bring this seamless experience to the GNOME App ecosystem.

Being a GNOME contributor, I'm already familiar with GNOME's Newcomer guidelines, development processes and the project's management tools (Gitlab, Hedgedoc, etc..) and resources as well as the project's coding style and community and, notably, the emphasis on diversity in the project.

My previous contributions demonstrate my commitment to the GNOME ecosystem as I've helped fill gaps in the desktop system. I wish to work on something that will have an echoing effect on many people while venturing into new fields and learning.

In general, my skills and determination for the project demonstrate that I'm the right person for this project.

How do you plan to achieve completion of your project?

I plan to work within the time frame of 360 hours, distributed over 15 weeks.

This timeframe will give me ample time to progress on each process step, accounting for miscellaneous activities like code review and implementing feedback.

I plan to work on each step individually, creating pull requests and ensuring that each step utilises the effort from the previous one, reducing unnecessarily complicated pull requests.

I am eager to use feedback in my work and iterate on each solution to finalise an MVP pull request.

Please provide a sequence of tasks and subtasks and how long (days) you estimate it will take you to complete each of them. Highlight important milestones/deliverables.

Period	Tasks
Before May 1	• Study the basics of the DAP protocol deeply • [DONE] Get GJS running locally • Compile Spidermonkey locally • Close my PR adding async Wasm support to GJS
May 1 - May 24	Community Bonding Period • Re-familiarise myself with GNOME's Code of Conduct. • Learn about GNOME's commit style.

	• Visit the GNOME Project Handbook to make sure I know all the information for GNOME Contributors. • Continue discussing with my mentor • Study the DAP spec deeply • Write a prototype <code>dap.js</code> that can respond to <code>`initialize`</code>, <code>launch</code> and <code>setBreakpoints</code> and emit a <code>`stopped`</code> event
May 25 - June 22	Month 1 • Implement DAP message framing (parsing, serializing) via stdio • Implement a few base lifecycle DAP events (initialize, launch, configurationDone, setBreakpoints, continue, stopped). • Create a simple VS Code extension to drive the DAP adapter
June 22 - July 27	Month 2 + Midterm evaluations • Add support for more DAP events (stackTrace, scopes, variables) • Ensure stack traces work (with source map support) • Add support for stepping through the code (stepIn, next, stepOut, finish) • Add exception handling (scoped, exception)
July 27 - August 24	Month 3 • Add support for evaluating custom commands (evaluate) • Integrate source map in responses • Wrap all MRs up
	Extras (If time permits) • Add support for activation using SIGUSR1, then via Unix socket • Add support for GNOME Shell to initiate the debugger via D-Bus

What are your past experiences with the open source world as a user and as a contributor?

I have been an open-source user for quite some time. Frustrated with the clunkiness and slowness of my Windows install on an ancient laptop, I got curious, explored other operating systems, and finally did my first Linux install with Ubuntu. Through time, I started distro hopping, and I must admit, I never settled. I've been using Alpine for a few months, but I always install GNOME on almost every distro I install, which makes me a very seasoned user.

GNOME fascinated me because of its different yet flexible user interface. When I joined, I'd call it the "app boom", where many apps were being developed for GNOME because of the shiny new

GTK4 and Libadwaita. When I learnt that countless volunteers were working behind the project, I became touched and decided to contribute something.

Because I liked Web Development, the only language I was familiar with was JavaScript. After learning it was possible to create GNOME apps using JavaScript with GJS, I iterated for a year before finally building my first app, Sticky Notes. I was pleasantly surprised that many people liked the app, and I became aware of the effect of my simple contribution. With time, I created two more GNOME apps, with one becoming an official GNOME Core app and I started interacting more with members of the GNOME community. It's been one of the most friendly and welcoming communities in my life, and that inspired me to contribute even more, creating issues for problems I met and working on pull requests to implement features I wanted.

Please include links to your code contributions which have already been merged, or to Gitlab merge requests for the issues you fixed for the project of your proposal or any other GNOME projects. This demonstrates your willingness to learn and familiarity with development workflow.

- https://gitlab.gnome.org/GNOME/gjs/-/merge_requests/842
- https://gitlab.gnome.org/GNOME/gjs/-/merge_requests/841
- https://gitlab.gnome.org/GNOME/gjs/-/merge_requests/1078 (Pending)

If available, please include links to any code you wrote for other open source projects.

- **D-Spy:** [Use new libadwaita 1.4 widgets \(!14\)](#)
- **GNOME Control Center:** [Draft: Restyle - gnome-system-monitor \(!182\)](#)
- **gi-docgen:** [use `color-scheme: dark` when in dark mode \(!188\)](#)
- **GJS:** [use `meson setup` instead of ambiguous `meson` \(!842\)](#)
- **GJS:** [Pretty-print null and undefined \(!841\)](#)
- **Libgweather:** [Add cities in Rwanda \(!286\)](#)

What other relevant projects have you worked on previously and what knowledge have you gained from working on them?

I've previously created the following applications for the GNOME Ecosystem:

- [Sticky Notes](#)
- [Decibels](#)
- [Muzika](#)

These applications are written for the GJS runtime, and while building them, I also learnt about the GObject APIs

What other time commitments, such as school work, exams, research, another job, planned vacation, etc? What are the dates for these commitments and how many hours a week do these commitments take?

I will take GSoC as a part-time commitment for almost four months (June, July and parts of May and August).

I will also be a part-time student at the African Leadership University, where we study for 2 hours twice a week, totalling 4 hours a week.

I will dedicate around 8 hours a day for three days a week to my GSoC project, totalling 24 hours weekly.